

09

Evaluation Metrics

Accuracy · Precision · Recall · F1 · Support

How to truly judge if a model is good — beyond just accuracy

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What this is	The scores that tell you how good a model is
Built from	The confusion matrix (4 simple counts)
Key lesson	Accuracy alone can fool you badly
The big four	Accuracy, Precision, Recall, F1-score
When	After training — to decide if the model is usable

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	Why Metrics Matter	Accuracy can lie to you
2	The Confusion Matrix	The 4 counts everything is built from
3	Accuracy	The simple but risky score
4	Precision	When you say YES, are you right?
5	Recall	Did you catch all the real cases?
6	Precision vs Recall	The trade-off, and which to choose
7	F1-Score	Balancing precision and recall
8	Support	What that last column means
9	The Full Report	Reading classification_report
10	Picking the Right Metric	Real situations and choices
11	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

3. Accuracy — Simple but Risky

Accuracy (■■■■■■■■■■ — ■■■■■■) is the most obvious score: out of all predictions, how many did the model get right?

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

In plain words: correct answers divided by total answers. If the model made 100 predictions and got 90 right, accuracy is 90%.

When Accuracy Is Fine

- When your classes are balanced — roughly equal numbers of YES and NO.
- When every type of mistake costs about the same.

When Accuracy Lies

- When data is **imbalanced** (■■■ — ■■■■ ■■ ■■■■ ■■■■). Like the disease example: 99% accuracy while catching zero sick people.
- When one kind of mistake is far worse than the other (missing cancer vs a false alarm).

■ *Rule: the more imbalanced your data, the less you should trust accuracy. Switch your attention to precision and recall instead.*

4. Precision — How Trustworthy is a YES?

Precision (■■■■■■■■■ — ■■■■■■■■■■) asks: of all the times the model said YES, how many were actually right? It measures how much you can trust a YES from the model.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Think of a spam filter. High precision means: when it marks an email as spam, it's almost always really spam. Low precision means it keeps flagging good emails as spam (annoying false alarms). Precision cares about avoiding **false positives**.

When to Care About Precision

- Spam detection — you don't want to lose real emails.
- Recommending products — a wrong suggestion wastes the user's trust.
- Any case where a false alarm is expensive or annoying.

5. Recall — Did We Catch Them All?

Recall (■■■■■ — ■■■■■■ ■■■■ ■■■■) asks: of all the real YES cases out there, how many did the model actually catch? It measures how good the model is at not missing things.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Think of cancer screening. High recall means: of all the people who truly have cancer, the model found almost all of them. A miss here (false negative) is dangerous — a sick person sent home. Recall cares about avoiding **false negatives**.

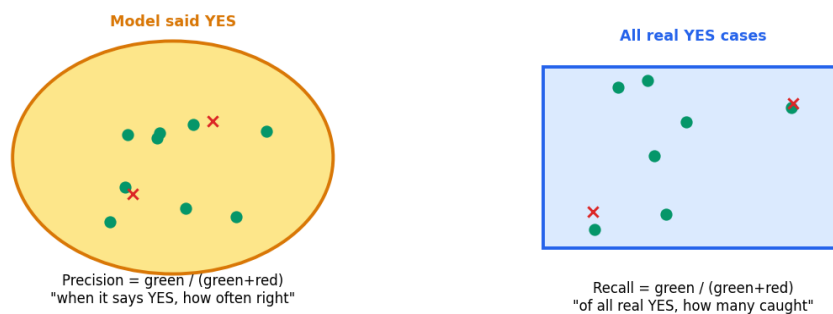


Fig 2 — Precision looks inside 'what the model flagged'. Recall looks inside 'all real cases'.

When to Care About Recall

- Disease detection — missing a sick patient is the worst outcome.
- Fraud detection — you want to catch every fraud, even with some false alarms.
- Safety systems — better a false alarm than a missed danger.

6. Precision vs Recall — The Trade-off

Here's the tricky part: precision and recall usually pull against each other. Push one up and the other tends to drop. You control the balance with the threshold from the last topic.

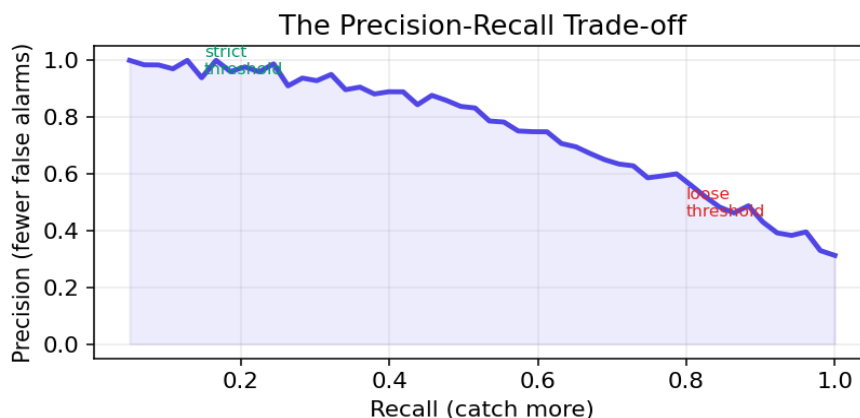


Fig 3 — As you catch more cases (higher recall), you usually make more false alarms (lower precision).

- **Strict** (high threshold): the model only says YES when very sure. High precision, low recall — few false alarms, but misses some real cases.
- **Loose** (low threshold): the model says YES easily. High recall, low precision — catches almost everything, but many false alarms.

There's no universally 'correct' choice — it depends on which mistake hurts more in your real problem. That decision is yours to make, not the model's.

7. F1-Score — The Balance

Sometimes you want a single number that balances precision and recall. That's the **F1-score**. It's a special kind of average (the harmonic mean) that only stays high when BOTH precision and recall are high.

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Why not a normal average? Because a normal average can be fooled. If precision is 1.0 and recall is 0.0, a normal average says 0.5 (looks okay). But F1 says 0.0 — correctly telling you the model is broken. F1 punishes a model that is great at one thing and terrible at the other.

Precision	Recall	Normal avg	F1-score
1.0	0.0	0.50	0.00
0.9	0.3	0.60	0.45
0.8	0.8	0.80	0.80
0.6	0.9	0.75	0.72

■ Use F1 when classes are imbalanced and you care about both precision and recall. It's the most common single-number score for imbalanced classification problems.

8. Support — The Count

Support (■■■■■■■■) is the simplest one. It's just the number of real examples of each class in your test data. It doesn't measure quality — it tells you how much data each score is based on.

Why it matters: a 100% recall based on just 3 examples is far less trustworthy than 90% recall based on 5000 examples. Support warns you when a score is built on very little data, so you don't over-trust it.

■ Always glance at support. If one class has tiny support, its precision/recall numbers can jump around wildly and shouldn't be taken too seriously.

9. The Full Report

sklearn gives you all of these at once with `classification_report`. Here's how to read it:

```
from sklearn.metrics import classification_report, confusion_matrix

print(confusion_matrix(y_test, preds))
print(classification_report(y_test, preds))

#           precision    recall  f1-score   support
#
#  class 0       0.92      0.95      0.93        100
#  class 1       0.88      0.82      0.85         60
#
# accuracy              0.90              160
# macro avg       0.90      0.88      0.89        160
# weighted avg    0.90      0.90      0.90        160
```

Row / Column	What it tells you
precision column	Trust level of each class's YES
recall column	How many of each real class were caught
f1-score column	Balance score per class
support column	How many real examples of each class
accuracy row	Overall correctness across everything
macro avg	Plain average across classes (treats all equal)
weighted avg	Average weighted by support (bigger class counts more)

10. Picking the Right Metric

A quick guide for real situations:

Situation	Best metric	Why
Balanced data, equal costs	Accuracy	Simple and fair here
Disease / fraud detection	Recall	Missing a case is dangerous
Spam filter	Precision	False alarms lose real emails
Imbalanced, care about both	F1-score	Balances the two fairly
Predicting a number	MSE / R ²	Those are regression metrics

Ask yourself one question: which mistake is worse for my problem — a false alarm or a miss? If false alarms are worse, focus on precision. If misses are worse, focus on recall. If both matter, use F1.

11. Common Mistakes

Mistake	Why it's a problem	Fix
Trusting only accuracy	Hides failure on rare classes	Check precision and recall too
Ignoring class imbalance	Accuracy looks falsely great	Use F1 or recall
Ignoring support	Tiny-data scores are shaky	Glance at the support column
Wrong metric for the goal	Optimising the wrong thing	Match metric to real cost
Judging on training data	Always looks too good	Measure on the test set
Forgetting the threshold	Default 0.5 may not fit	Tune it for precision/recall

Quick Reference Cheatsheet

Metric / Task	Code
Confusion matrix	<code>confusion_matrix(y_test, preds)</code>
Full report	<code>classification_report(y_test, preds)</code>
Accuracy	<code>accuracy_score(y_test, preds)</code>
Precision	<code>precision_score(y_test, preds)</code>
Recall	<code>recall_score(y_test, preds)</code>
F1-score	<code>f1_score(y_test, preds)</code>
Precision (multi-class)	<code>precision_score(y, p, average='macro')</code>
Per-class scores	<code>precision_score(y, p, average=None)</code>

The four formulas to remember:

$$\begin{aligned}\text{Accuracy} &= (TP + TN) / \text{all} & \text{Precision} &= TP / (TP + FP) \\ \text{Recall} &= TP / (TP + FN) & \text{F1} &= 2 \times P \times R / (P + R)\end{aligned}$$

Next Topic: XGBoost — a powerful and hugely popular model that wins many competitions. We'll see how it combines many small trees into one strong predictor.